



VIT-AP UNIVERSITY

School of Computer Science and Engineering

Data Structures and Algorithms

Lab Assignment

Experiment 5

Lab Task-5: Searching Techniques & Graphs

Name : Abhinav Sai

Register Number : 24BCA7148

Slot : L31 + L32 + L43 + L44

Professor : Dr. Madala Gargi

University : VIT-AP University

Date: June 05, 2026

Table of Contents

S.No.	Program Title	Description
1	Program 1	Linear Search Using Sample Data
2	Program 2	Binary Search Using Sample Data
3	Program 3	Breadth First Search Traversal of a Graph
4	Program 4	Depth First Search Traversal of a Graph

Program 1: Linear Search Using Sample Data

Aim:

Implement linear search and find a target value by checking each element sequentially.

Theory:

Linear search is the simplest searching technique.

It scans each element one by one until the target is found or the list ends.

Its worst-case time complexity is $O(n)$.

Algorithm:

1. Read the array and the target element.
2. Start from the first element and compare it with the target.
3. If the target matches, report the position and stop.
4. If the end of the array is reached, report that the element is not found.

Program Code:

```
public class LinearSearchLab {
    static int linearSearch(int[] arr, int target) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == target) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {12, 5, 19, 8, 25, 7};
        int target = 19;

        System.out.print("Array: ");
        for (int value : arr) {
            System.out.print(value + " ");
        }
        System.out.println();

        int index = linearSearch(arr, target);
        if (index != -1) {
            System.out.println("Element " + target + " found at position " + (index
+ 1));
        }
    }
}
```

```
        } else {  
            System.out.println("Element " + target + " not found");  
        }  
    }  
}
```

Output:

```
Array: 12 5 19 8 25 7  
Element 19 found at position 3  
Name: Abhinav Sai  
Register Number: 24BCA7148
```

Result:

Linear search was implemented and the target element was located by sequential comparison.

Program 2: Binary Search Using Sample Data

Aim:

Implement binary search on a sorted array using divide and conquer to locate the target element.

Theory:

Binary search works only on sorted data.

It repeatedly divides the search space into half based on the middle element.

Its worst-case time complexity is $O(\log n)$.

Algorithm:

1. Take a sorted array and the target element.
2. Find the middle element of the current search range.
3. If the target equals the middle element, return the position.
4. Otherwise continue searching in the left or right half until the target is found or the range becomes empty.

Program Code:

```
public class BinarySearchLab {
    static int binarySearch(int[] arr, int target) {
        int low = 0;
        int high = arr.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (arr[mid] == target) {
                return mid;
            } else if (arr[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] arr = {3, 8, 12, 19, 25, 31, 42};
        int target = 25;

        System.out.print("Sorted array: ");
```

```
        for (int value : arr) {
            System.out.print(value + " ");
        }
        System.out.println();

        int index = binarySearch(arr, target);
        if (index != -1) {
            System.out.println("Element " + target + " found at position " + (index
+ 1));
        } else {
            System.out.println("Element " + target + " not found");
        }
    }
}
```

Output:

```
Sorted array: 3 8 12 19 25 31 42
Element 25 found at position 5
Name: Abhinav Sai
Register Number: 24BCA7148
```

Result:

Binary search was implemented on sorted sample data and the target was found efficiently.

Program 3: Breadth First Search Traversal of a Graph

Aim:

Traverse a graph using breadth first search and display the visiting order of vertices.

Theory:

Breadth first search explores a graph level by level.

It uses a queue to visit all adjacent vertices before moving deeper into the graph.

BFS is useful for shortest path discovery in unweighted graphs.

Algorithm:

1. Start from the chosen source vertex and mark it as visited.
2. Insert the source into a queue.
3. Remove a vertex from the queue, print it, and enqueue all unvisited adjacent vertices.
4. Repeat until the queue becomes empty.

Program Code:

```
import java.util.LinkedList;
import java.util.Queue;

public class BFSGraphLab {
    static void bfs(int[][] graph, int start) {
        boolean[] visited = new boolean[graph.length];
        Queue<Integer> queue = new LinkedList<>();

        visited[start] = true;
        queue.add(start);

        while (!queue.isEmpty()) {
            int vertex = queue.poll();
            System.out.print((vertex + 1) + " ");

            for (int i = 0; i < graph.length; i++) {
                if (graph[vertex][i] == 1 && !visited[i]) {
                    visited[i] = true;
                    queue.add(i);
                }
            }
        }
    }
}
```

```
public static void main(String[] args) {  
    int[][] graph = {  
        {0, 1, 1, 0, 0},  
        {1, 0, 0, 1, 1},  
        {1, 0, 0, 0, 1},  
        {0, 1, 0, 0, 0},  
        {0, 1, 1, 0, 0}  
    };  
  
    System.out.print("BFS Traversal: ");  
    bfs(graph, 0);  
    System.out.println();  
}  
}
```

Output:

```
BFS Traversal: 1 2 3 4 5  
Name: Abhinav Sai  
Register Number: 24BCA7148
```

Result:

Breadth first search was used to traverse the graph level by level and print the order.

Program 4: Depth First Search Traversal of a Graph

Aim:

Traverse a graph using depth first search and display the visiting order of vertices.

Theory:

Depth first search explores as far as possible along each branch before backtracking.

It is commonly implemented using recursion or a stack.

DFS is useful for path finding, cycle detection, and topological exploration.

Algorithm:

1. Start from the chosen source vertex.
2. Mark the current vertex as visited and print it.
3. Recursively visit every unvisited adjacent vertex.
4. Backtrack when no unvisited adjacent vertices remain.

Program Code:

```
public class DFSGraphLab {
    static void dfs(int[][] graph, boolean[] visited, int vertex) {
        visited[vertex] = true;
        System.out.print((vertex + 1) + " ");

        for (int i = 0; i < graph.length; i++) {
            if (graph[vertex][i] == 1 && !visited[i]) {
                dfs(graph, visited, i);
            }
        }
    }

    public static void main(String[] args) {
        int[][] graph = {
            {0, 1, 1, 0, 0},
            {1, 0, 0, 1, 1},
            {1, 0, 0, 0, 1},
            {0, 1, 0, 0, 0},
            {0, 1, 1, 0, 0}
        };

        boolean[] visited = new boolean[graph.length];
        System.out.print("DFS Traversal: ");
        dfs(graph, visited, 0);
    }
}
```

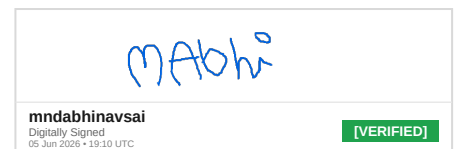
```
        System.out.println();  
    }  
}
```

Output:

```
DFS Traversal: 1 2 4 5 3  
Name: Abhinav Sai  
Register Number: 24BCA7148
```

Result:

Depth first search was implemented to traverse the graph using recursive exploration.



Document Information

Document Name: VIT_AP_DSA_Lab_Assignment_Lab_Task_5_v1.pdf

Document ID: 6a230cdf4d81730dd1b2926

Sender: Abhi (mndabhinavsai@gmail.com)

Completed Date: Fri, 05 Jun 2026 19:49:15 GMT

[VERIFIED] | [TAMPER PROTECTED] | [LEGALLY BINDING] | [AUDIT TRAIL COMPLETE]

Identity Verification

Name: Signer

Email: mndabhinavsai@gmail.com

Event Information

Date/Time: Fri, 05 Jun 2026 19:10:51 GMT

IP Address: ::1

Location: Unavailable

Device Information

Browser: Chrome

OS: Windows

Device: Desktop



Scan to Verify

Signature ID: SIG-2026-8E4C40 | Audit ID: AUD-EE98D561

Fingerprint (SHA-256): 961d205d072b319fe175e080939874c8ed174a7d6dc52153bd442677e68676a1